

UNIVERSITÀ DEGLI STUDI MILANO-BICOCCA

APPUNTI DEL CORSO DI

Analisi e Progettazione del Software

Scritto da:

Nicholas Scorrano

Anno Accademico 2025/2026

Indice

1	Pattern GRASP	2
1.1	Definizione	2
1.1.1	Approccio RDD	2
1.1.2	Principi fondamentali	2
1.1.3	Relazione tra gli elaborati	3
1.2	Rappresentazione di un Pattern GRASP	3
2	Pattern GRASP di base	4
2.1	Creator	4
2.1.1	Problema	4
2.1.2	Soluzione	4
2.1.3	Benefici	4
2.1.4	Controindicazioni	4
2.1.5	Flusso di lavoro logico	5
2.1.6	Modello di dominio	5
2.1.7	Modello di sequenza	5
2.1.8	Diagrammi delle classi di progetto	6
2.2	Information Expert	6
2.2.1	Problema	6
2.2.2	Soluzione	6
2.2.3	Benefici	6
2.2.4	Controindizioni	7
2.2.5	Flusso di lavoro	7
2.2.6	Diagramma di sequenza	7
2.2.7	Diagramma delle classi	7
2.3	Low Coupling	8
2.3.1	Problema	8
2.3.2	Soluzione	8
2.3.3	Benefici	8
2.3.4	Controindicazioni	8
2.3.5	Flusso di lavoro	8
2.4	High Cohesion	9
2.4.1	Problema	9
2.4.2	Soluzione	9
2.4.3	Benefici	9
2.4.4	Controindicazioni	9
2.4.5	Flusso di lavoro	10
2.5	Controller	10
2.5.1	Problema	10
2.5.2	Soluzione	10
2.5.3	Benefici	11
2.5.4	Controindicazioni	11
2.5.5	Flusso di lavoro	11

1. Pattern GRASP

1.1 Definizione

I pattern GRASP (General Responsibility Assignment Software Pattern) sono un insieme di principi fondamentali e linee guida utilizzati nella **progettazione del software orientata agli oggetti** per supportare il progettista nell'assegnazione delle responsabilità a classi e oggetti.

1.1.1 Approccio RDD

Più che schemi di codice preconfezionati, i GRASP deniscano un approccio metodologico noto come **Responsability-Driven Design (Progettazione Guidata alla responsabilità)** formalizzando sotto forma di pattern le regole e le soluzioni idiomatiche applicate.

1.1.2 Principi fondamentali

- **Focus sulle Responsabilità:**

Nella progettazione a oggetti, i requisiti espressi nei casi d'uso vengono soddisfatti traducendoli in responsabilità per le classi del software.

Tali responsabilità si dividono in 2 macro-categorie:

- **Responsabilità di FARE (Doing):** Come eseguire un'azione di business, creare un altro oggetto o coordinare attività.
- **Responsabilità di CONOSCERE (Knowing):** Come la conoscenza dei propri dati privati incapsulati, delle relazioni o di elementi calcolabili.

- **Applicazione Dinamica:**

Lo strumento principale in cui si applicano e si concretizzano i pattern GRASP è il disegno dei **diagrammi di interazione UML**.

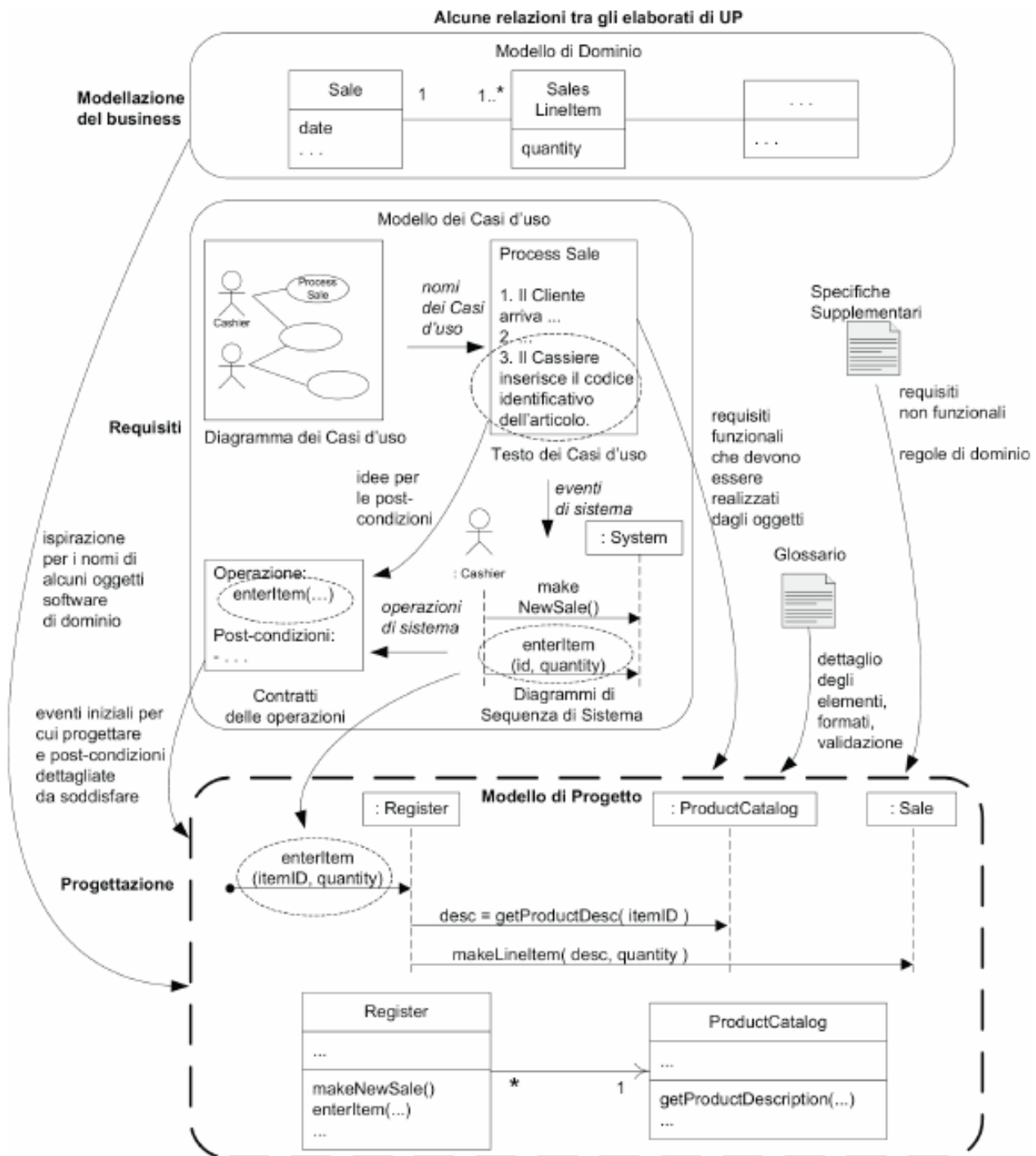
È proprio nel momento in cui si definiscono i messaggi scambiati tra gli oggetti che il progettista sceglie a quale entità assegnare una specifica funzione.

- **Obiettivo Qualificativo:**

L'applicazione metodica di questi pattern mira a massimizzare metriche software essenziali, riducendo l'impatto dei cambiamenti futuri e migliorando la manutenibilità.

I due capisaldi di questo obiettivo sono il raggiungimento di un **accoppiamento basso**(Low Coupling) e di una **coesione alta**(High Cohesion)

1.1.3 Relazione tra gli elaborati di UP



1.2 Rappresentazione di un Pattern GRASP

Nome del pattern: Information Expert

Problema: Qual è un principio di base con cui assegnare responsabilità agli oggetti?

Soluzione: Assegna una responsabilità alla classe che ha le informazioni necessarie per soddisfarla.

2. Pattern GRASP di base

2.1 Creator

Nome:	Creator (Creatore)
Problema:	Chi crea un oggetto A?
Soluzione	Assegna alla classe B la responsabilità di creare un'istanza della classe A se una delle seguenti condizioni è vera (più sono vere, meglio è):
(può essere	• B “contiene” o aggrega con una composizione oggetti di tipo A. • B registra A. • B utilizza strettamente A. • B possiede i dati per l'inizializzazione di A.
vista come un consiglio):	

2.1.1 Problema

In un softwaread oggetti, la creazione di nuovi oggetti è un'operazione pervasiva.

Tuttavia, se una classe crea direttamente un'altra classe,diventa fortemente accoppiata a quest'ultima. Se la classe creata **cambia o viene sostituita**, anche la classe che la crea potrebbe subire modifiche.

Il problema è quindi: Come assegnare la responsabilità di creazione in modo da non aumentare l'accoppiamento complessivo al sistema?

2.1.2 Soluzione

Il pattern Creator stabilisce che una classe B dovrebbe ricevere la responsabilità di creare un'istanza della classe A se si verifica almeno una (e preferibilmente più di una) delle seguenti condizioni:

1. **Aggregazione/Composizione:** B contiene, aggrega o racchiude gli oggetti di tipo A
2. **Registrazione:** B registra o tiene traccia delle istanze di A
3. **Uso Stretto:** B utilizza strettamente ed estesamente gli oggetti di tipo A
4. **inizializzazione:** B possiede i dati di inizializzazione che devono essere passati ad A al momento della sua creazione

2.1.3 Benefici

- Basso Accoppiamento
- Incapsulamento

2.1.4 Controindicazioni

Non sempre il pattern Creator è la scelta migliore.

Esistono situazioni in cui l'applicazione rigida di questo principio crea problemi:

- **Logica di creazione complessa:**

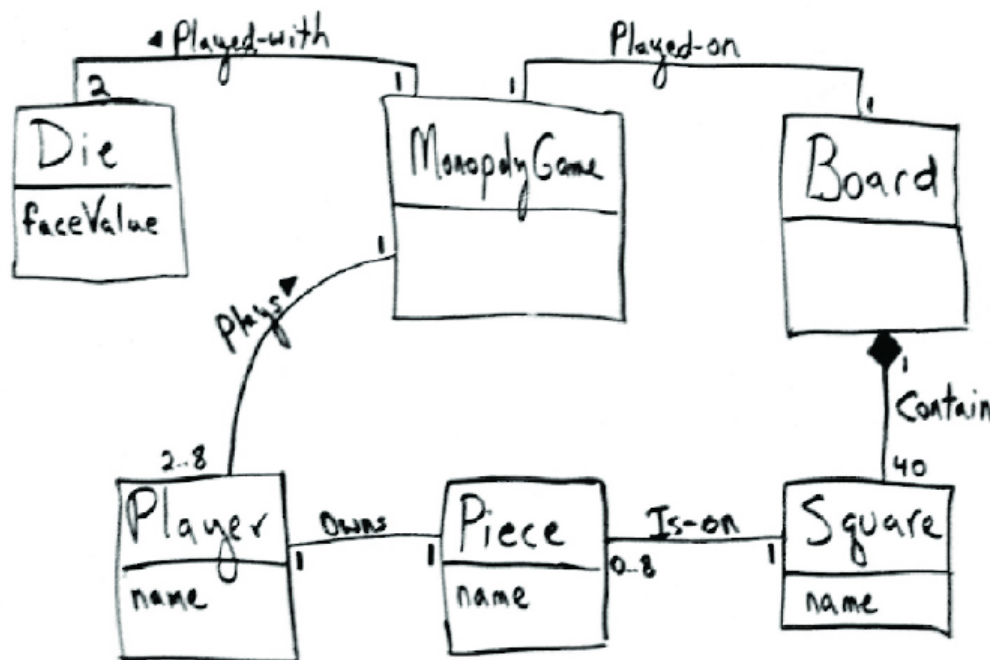
Se la creazione richiede passaggi condizionali intricati, la scelta dinamica di sottoclassi o l'interfaciamento con librerie esterne, assegnare la creazione alla classe aggregatrice potrebbe distruggere la coesione

- **Riuso del processo di creazione:** Se lo stesso identico processo di creazione serve a molte classi diverse

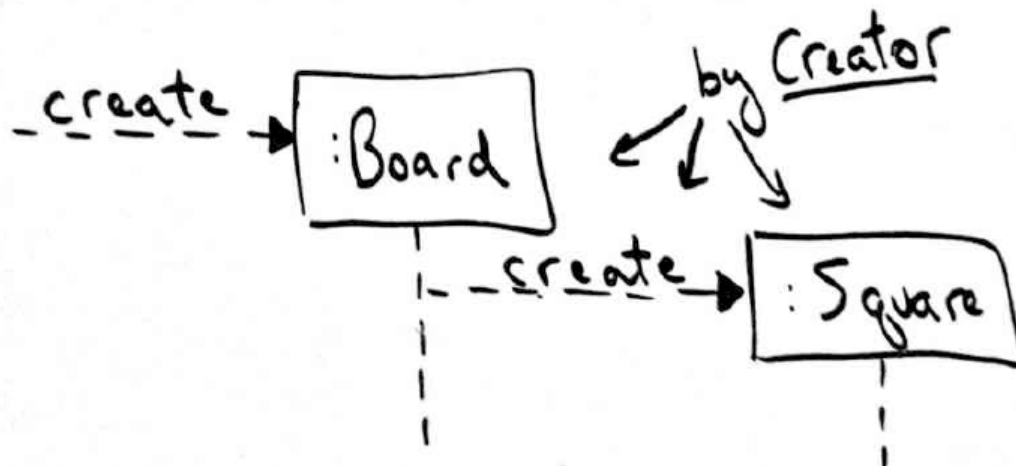
2.1.5 Flusso di lavoro logico

1. Leggi il contratto dell'operazione di sistema (che ti dice cosa deve essere creato)
2. Nel diagramma di sequenza si delinea chi crea cosa
3. Si riporta la funzione e la freccia di dipendenza nel diagramma delle classi

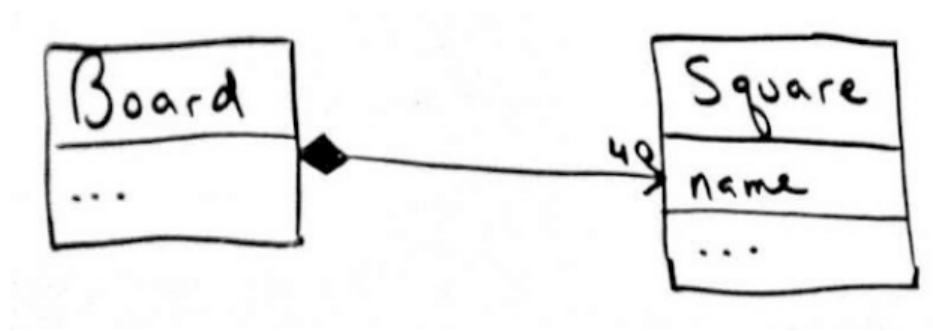
2.1.6 Modello di dominio



2.1.7 Modello di sequenza



2.1.8 Diagrammi delle classi di progetto



2.2 Information Expert

Nome del pattern: Information Expert (Esperto dell'Informazione)

Problema: Qual è un principio di base fondamentale con cui assegnare le responsabilità agli oggetti?

Soluzione: Assegna una responsabilità alla classe che possiede le informazioni necessarie per soddisfarla.

2.2.1 Problema

Nello sviluppo software, quando si riscrive un nuovo caso d'uso, ci si trova davanti a decine di frammenti di logica di business.

Se queste responsabilità vengono assegnate a caso, si rischia di scivolare nell'approccio procedurale, dove le classi di dominio diventano dei semplici contenitori anemidici di dati, mentre tutta la logica operativa viene accentrata in un unico enorme blocco. Questo porta a un codice di bassa qualità

Information Expert risolve questo caso fornendo un criterio oggettivo: esamina i dati e mette l'azione dove risiedono i dati necessari a compierla

2.2.2 Soluzione

La direttiva è semplice: guarda quale classe possiede le informazioni richieste per completare il compito e assegna il metodo a quella classe.

Le "informazioni" necessarie possono essere:

- Attributi privati della classe stessa
- Oggetti strettamente correlati ad essa
- Informazioni che la classe può calcolare o derivare interrogando i suoi componenti

Se le informazioni sono distribuite su più classi, l'information Expert diventa un concetto incrementale: si crea una catena di deleghe in cui ogni classe fa la sua parte basandosi su ciò che conosce direttamente.

2.2.3 Benefici

- Mantenimento dell'incapsulamento
- Alta Coesione
- Basso Accoppiamento

- Facilità di manutenzione

2.2.4 Controindizioni

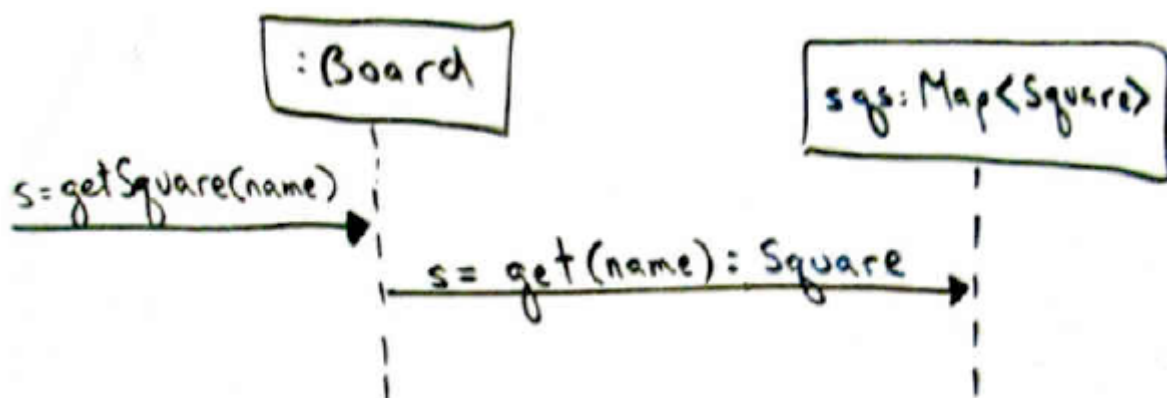
- Problemi di persistenza
- Logiche di visualizzazione (UI):

Chi conosce i dati dell'utente? La classe User. Ma non dobbiamo permettere a User di stampare se stessa in HTML o di aprire una finestra pop-up grafica. La responsabilità grafica va separata nettamente.

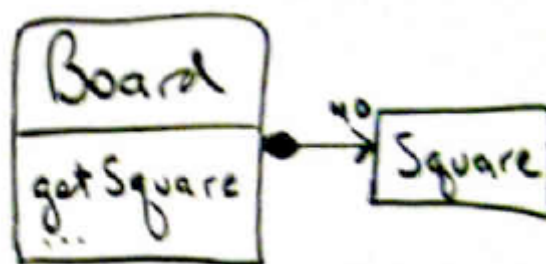
2.2.5 Flusso di lavoro

1. **Identifica il requisito:** Leggi i contratti delle operazioni o i casi d'uso
2. **Cerca le informazioni:**
Guarda il modello di dominio e individua quali classi possiedono gli attributi necessari a quel calcolo
3. **Inizia dall'alto:** Trova la classe che fa da aggregatore principale e assegna il metodo di alto livello
4. **Applica la delega:**
Se la classe principale non ha tutti i dettagli, scendi di un livello nell'albero delle associazioni e assegna sotto-responsabilità alle classi componenti
5. **Disegna l'interazione:**
Convalida questa scelta tracciando i messaggi nel diagramma di sequenza.
Se le frecce fluiscono in modo lineare seguendo la struttura dei dati, l'applicazione dell'information Expert è corretta

2.2.6 Diagramma di sequenza



2.2.7 Diagramma delle classi



2.3 Low Coupling

Nome del pattern: **Low Coupling** (Accoppiamento Basso)

Problema: Come ridurre l'impatto dei cambiamenti, supportare un alto riuso e facilitare la manutenzione?

Soluzione: Assegna le responsabilità in modo tale che l'accoppiamento (le dipendenze tra le classi) rimanga il più basso possibile.

2.3.1 Problema

L'accoppiamento è la misura di quanto fortemente un elemento è connesso a, conosce o dipende da altri elementi.

Se un sistema ha un alto accoppiamento, le classi sono strettamente interconnesse tra loro in una rete ingarbugliata.

Questo approccio comporta 3 gravi problemi:

- **Effetto Dominio:** Una modifica apparentemente innocua in una classe rompe il funzionamento di altre 10 classi sparse nel codice
- **Impossibilità di riuso**
- **Difficoltà di comprensione**

2.3.2 Soluzione

Il pattern suggerisce di valutare costantemente le alternative di progettazione e di scegliere sempre la soluzione che minimizza i legami di dipendenza.

In un linguaggio orientato agli oggetti, la classe A è accoppiata alla classe B se:

- A ha un attributo che fa riferimento a un istanza di B
- A invoca i metodi di un oggetto B
- A ha un metodo che riceve come argomento o restituisce un oggetto B
- A è una sottoclasse di B

2.3.3 Benefici

- **Isolamento delle modifiche**
- **Elevato riuso**
- **Testabilità semplificata**

2.3.4 Controindicazioni

- **Metodi ad-hoc gonfiati**
- **Infrastrutture e standard stabili**

2.3.5 Flusso di lavoro

1. Abbozza le alternative:

Quando devi implementare un messaggio di sistema pensa ad almeno due modi diversi in cui gli oggetti potrebbero cooperare per raggiungere il risultato

2. Conta le linee di dipendenza:

Guarda le frecce del tuo diagrama di interazione.

Quanti oggetti diversi deve conoscere la classe di partenza per portare a termine il compito?

3. Valuta la stabilità: Se un accoppiamento è inevitabile, cerca di indirizzarlo verso l'oggetto più stabile

4. Trova il compromesso con la coersione:

Assicurati che l'alternativa scelta per ridurre l'accoppiamento non distrugga l'alta coesione della classe ricevente

2.4 High Cohesion

Nome del pattern: **High Cohesion** (Coesione Alta)

Problema: Come mantenere gli oggetti focalizzati, comprensibili e gestibili, supportando al contempo un basso accoppiamento?

Soluzione: Assegna le responsabilità in modo che le operazioni interne di una classe siano strettamente correlate dal punto di vista funzionale.

2.4.1 Problema

Nello sviluppo di sistemi complessi, c'è la forte tentazione di inserire nuove funzioni all'interno delle poche classi già esistenti e note, creando involontariamente delle "classi mostro".

Se una classe ha una **bassa coesione**, significa che sta svolgendo troppi compiti eterogenei che non c'entrano nulla l'uno con l'altro.

Questo scenario comporta gravi problemi:

- **Fragilità e difficoltà di lettura**
- **Scarsa riusabilità**
- **Saturazione da cambiamento:** La classe sarà costretta a cambiare ogni volta che cambia uno qualsiasi dei cinque requisiti di business differenti di cui si fa carico

2.4.2 Soluzione

Il pattern suggerisce di **delegare** i compiti specialistici e mantenere ogni classe **specializzata** in un unico blocco logico di funzionalità. Una classe ad alta coesione fa "una sola cosa fatta bene".

2.4.3 Benefici

- **Facilità di comprensione**
- **Manutenzione localizzata**
- **Sviluppo in parallelo**

2.4.4 Controindicazioni

L'eccesso opposto della bassa coesione è la frammentazione esasperata, se si applica il principio in modo paranoico, si rischia di creare centinaia di microscopiche classi formate da un solo metodo e tre righe di codice ciascuna.

Questo approccio non fa altro che nascondere la complessità del sistema dentro un labirinto di collegamenti e chiamate, peggiorando drasticamente l'accoppiamento del software e rendendo il debugging un incubo.

Una coesione moderatamente alta è il target ideale.

2.4.5 Flusso di lavoro

Per garantire un'alta coesione durante le sessioni di progettazione nei diagrammi di sequenza e delle classi:

1. **Definisci l'identità della classe**
2. **Monitora i metodi aggiunti**
3. **Ispezione le variabili di istanza**
4. **Applica la delega attiva**

2.5 Controller

Nome del pattern:	Controller (Controllore)
Problema:	Qual è il primo oggetto oltre lo strato della UI che riceve e coordina l'esecuzione di un'operazione di sistema?
Soluzione:	Assegna questa responsabilità a un oggetto che rappresenti l'intero sistema, un dispositivo logico, o lo scenario del caso d'uso corrente.

2.5.1 Problema

Quando un utente interagisce con un'interfaccia grafica, l'interfaccia cattura un evento software.

Se il codice che esegue l'operazione di business viene scritto direttamente dentro il codice del pulsante grafico, si commette un grave errore di progettazione.

Questo approccio genera 2 problemi principali:

- **Dipendenza dalla tecnologia visiva**
- **Perdita di controllo e coesione**

Il controller risolve questo problema fungendo da intermediario e coordinatore

2.5.2 Soluzione

Il pattern stabilisce che la UI deve limitarsi a catturare il clic dell'utente e delegarlo direttamente a un oggetto specifica che sta ne background, chiamato appunto controller.

Il GRASP suggerisce due modi principali per scegliere o creare un oggetto controller:

- **Facade Controller**
Si assegna la responsabilità a un oggetto che rappresenta il sistema complessivo, il software nel suo insieme o il dispositivo fisico in uso
- **Use-Case Controller**
Si crea un oggetto che rappresenta l'intero scenario o la sequenza di azioni di un singolo caso d'uso

2.5.3 Benefici

- Disaccoppiamento della UI
- Riutilizzo dei canali di input
- Facilità di test

2.5.4 Controindicazioni

Il rischio più comune quando si implementa questo pattern è la creazione di un Controllore Gonfio.

2.5.5 Flusso di lavoro

1. **Analizza i diagrammi di sequenza di sistema:** Identifica quali sono messaggi che l'attore invia al sistema
2. **Scegli il tipo di controller**
3. **Piazza il blocco nel diagramma di interazione:** Nei diagrammi di sequenza dettagliati, inserisci la linea di vita della UI alla sinistra e, come secondo oggetto subito a destra, inserisci l'oggetto Controller
4. **Disegna la delega:** Fai partire la freccia dalla UI al Controller. Subito dopo, fai partire le frecce dal Controller verso le vere classi del Modello di Dominio applicando l'Information Expert.